

Normalización: de una tabla plana a un modelo limpio

Ya tienes un modelo con tablas, llaves primarias y relaciones. Pero diseñar no es solo dibujar cajitas y flechas: hay una pregunta concreta que todavía no respondimos. ¿Cómo sabes si lo que diseñaste está bien hecho?

Las formas normales son la respuesta. Son tres reglas que, aplicadas en orden, te permiten verificar que tu base de datos no tiene datos duplicados, no genera errores al actualizarse y va a seguir siendo confiable sin importar cuántas transacciones acumule con el tiempo. A ese proceso de aplicar las reglas se le llama **normalización**.

¿Por qué no alcanza con "tener todo en una tabla"?

Antes de ver las reglas, vale entender el problema que resuelven.

Imagina que recibes este Excel con ventas:

Pedido ID	Cliente	Ciudad	Producto	Precio	Producto 2	Precio 2
101	María	Buenos Aires	Laptop HP	1.350	Mouse Logitech	28
102	Juan	Bogotá	Laptop HP	1.280		

A simple vista parece funcional. Pero hay varios problemas escondidos:

- La Laptop HP aparece con dos precios distintos. ¿Cuál es el correcto?
- Si María se cambia de ciudad, ¿en cuántos registros hay que actualizar ese dato?
- Si un pedido tiene diez productos, necesitas diez pares de columnas. Y la mayoría van a quedar vacías.
- "Buenos Aires" y "Bs. As." son la misma ciudad, pero la base las va a tratar como dos diferentes.

Cada uno de estos problemas tiene un nombre formal y una solución concreta. Las tres formas normales son exactamente eso: una regla para cada tipo de problema.

Primera Forma Normal (1FN): una celda, un solo valor

El problema que resuelve: grupos de columnas que se repiten para registrar múltiples valores del mismo tipo.

Mira de nuevo la tabla de arriba. Hay una columna "Producto" y una columna "Producto 2". Si el pedido tiene tres productos, necesitas "Producto 3". Si tiene diez, diez columnas. Y cuando un pedido solo tiene uno, las otras nueve quedan vacías. Eso no es solo desperdicio de espacio: hace imposible hacer consultas simples como "¿cuántas veces se vendió la Laptop HP?".

La regla de la 1FN es simple: **cada celda contiene un solo valor, sin grupos ni listas.**

La solución es separar los pedidos de su contenido en dos tablas: una con los datos generales del pedido y otra con cada producto vendido en su propia fila.

Pedido ID	Producto	Cantidad	Precio
101	Laptop HP	1	1.350
101	Mouse Logitech	1	28
102	Laptop HP	1	1.280

Ahora cada fila es un hecho concreto. Sin vacíos, sin columnas que se multiplican. Pero al hacer esto, aparece un problema nuevo que la siguiente regla resuelve.

Segunda Forma Normal (2FN): cada dato depende de toda la llave

El problema que resuelve: datos que solo dependen de una parte de la llave primaria, no de toda.

En la tabla de detalle de pedido que acabas de crear, la llave primaria es la combinación de **Pedido ID + Producto ID**: juntos identifican de forma única cada fila. Hasta ahí bien.

El problema aparece cuando agregas el nombre del producto y su precio de lista a esa misma tabla. Esos datos solo dependen del Producto ID, no del pedido. Resultado: el mismo producto queda registrado con información distinta según el pedido.

Pedido ID	Producto ID	Nombre	Precio lista	Precio venta
101	PO1	Laptop HP	1.350	1.350
102	PO1	Laptop HP	1.280	1.280

¿Cuál es el precio real de la Laptop HP? No hay forma de saberlo. Y si mañana cambias el nombre del producto, tienes que actualizarlo en cada pedido donde aparezca. Si se te olvida uno, la base queda inconsistente.

La regla de la 2FN es: **cada columna debe depender de la llave completa, no de una parte.**

La solución es mover el nombre y el precio de lista a su propia tabla de Productos. En el detalle del pedido solo queda la cantidad y el **precio al momento de la venta**, que sí depende del pedido específico porque puede variar por descuentos o promociones.

Con esto, el nombre del producto vive en un solo lugar. Si cambia, lo actualizas una vez y listo. Pero todavía hay una dependencia más escondida que resolver.

Tercera Forma Normal (3FN): nada depende de algo que no sea la llave

El problema que resuelve: campos que dependen de otro campo en lugar de depender de la llave primaria.

Mira la tabla de clientes:

Cliente ID	Nombre	Ciudad	Código postal
C01	María	Buenos Aires	1000
C02	Juan	Bogotá	11011

El código postal depende de la ciudad, no del cliente. Si María se cambia de ciudad, hay que actualizar el código postal en todos sus registros. Si hay un error de tipeo en la ciudad, el código postal queda asociado a algo que no existe.

A esto se le llama dependencia transitiva: un campo depende de otro campo que no es la llave. La regla de la 3FN es: **nada debe depender de algo que no sea la llave primaria.**

La solución es crear una tabla separada de Códigos Postales, relacionada con Ciudad. La tabla de clientes guarda solo el código postal como referencia, sin repetir la lógica de a qué ciudad pertenece.

El error que ninguna forma normal menciona, pero que también rompe todo

Además de las dependencias, en los datos de ejemplo apareció algo que parece menor pero no lo es:

Ciudad
Buenos Aires
Bs. As.

Son la misma ciudad escritas de dos formas distintas. Para la base de datos, son dos ciudades diferentes. Cualquier reporte que agrupe por ciudad va a mostrar resultados separados para cada una. Al tener una tabla de referencia para ciudades o códigos postales, este problema desaparece: solo existe una versión válida de cada valor.

¿No es más complicado tener siete tablas que una?

Es la duda más común y tiene sentido. Una tabla parece más simple que siete.

Pero esa simplicidad es aparente. Una tabla gigante con todo junto parece fácil al principio y se vuelve un problema real en cuanto los datos crecen o alguien necesita actualizarlos. Siete tablas bien relacionadas son más fáciles de mantener, más rápidas de consultar y mucho menos propensas a errores, porque cada dato vive en un solo lugar.

El modelo que venías construyendo en las clases anteriores ya cumplía con la mayoría de estas reglas. La única tabla nueva que se agregó en esta clase fue **Códigos Postales**, necesaria para eliminar la dependencia entre Cliente y Ciudad. Las otras seis ya existían.

Tabla	Qué resuelve
Pedidos	Datos generales del pedido sin repetir info del cliente o producto
Detalle de Pedido	Cada producto vendido en su propia fila, con precio del momento
Clientes	Datos del cliente en un solo lugar
Productos	Nombre y precio de lista en un solo lugar
Empleados	Datos del empleado sin mezclarlos con los pedidos
Códigos Postales	Elimina la dependencia entre cliente y ciudad
Categorías	Permite agrupar y analizar productos por tipo

Las tres reglas en una sola vista

Forma Normal	Qué elimina	La regla en una frase

1FN	Grupos de columnas repetidos	Una celda, un solo valor
2FN	Dependencias parciales	Cada columna depende de la llave completa
3FN	Dependencias transitivas	Nada depende de algo que no sea la llave primaria

Ahora que el modelo está validado con estas reglas, el siguiente paso es llevarlo a código. Ya sabes qué tablas necesitas y qué va en cada una. Lo que sigue es escribir las instrucciones que las crean de verdad dentro de un motor de base de datos, y para eso primero hay que entender con qué tipo de dato se define cada columna.